# Modular EV Data Logger Dashboard Framework (MEDDF)

## Frontend Configuration-Driven Dashboard Engine

---

# 1.Project Overview

## 1.1 Introduction

The Modular EV Data Logger Dashboard Framework (MEDDF) is a configuration-driven frontend dashboard engine designed to visualize EV telemetry data dynamically.

The backend system already exists and provides telemetry data through REST APIs and WebSocket. This project focuses entirely on building a scalable and dynamic frontend rendering engine.

---

## 1.2 Objective

The objective of this project is to:

- Build a dynamic dashboard rendering engine
- Avoid hardcoded dashboards
- Support multiple vehicle types
- Enable UI changes via configuration
- Handle real-time telemetry updates efficiently
- Ensure scalability and performance

---

# 1.3 Problem Statement

Traditional EV dashboard systems are often hardcoded for specific vehicle types.
Whenever a new vehicle model is introduced, developers must manually modify UI components and rendering logic. This leads to:

- Code duplication
- Poor scalability
- High maintenance cost
- Increased development time
- Reduced flexibility

Additionally, real-time telemetry dashboards require optimized rendering to avoid performance degradation when frequent data updates occur.

Therefore, there is a need for a scalable, configuration-driven frontend engine that:

- Dynamically renders dashboards
- Supports multiple vehicle types
- Avoids hardcoded layouts
- Ensures efficient real-time updates
- Maintains clean architecture

The MEDDF project addresses these challenges by implementing a configuration-driven rendering system using modern frontend technologies.

# 2. Technology Stack

## Frontend Framework

- React (with Vite)

## State Management

- Zustand

## API Communication

- Axios (REST API)

## Real-Time Communication

- Native WebSocket API

## Styling

- Tailwind CSS
- CSS Grid

## Architectural Patterns Used

- Configuration-Driven Architecture
- Factory Pattern (Tile Registry)
- Observer Pattern (Selective Re-render)
- Separation of Concerns

---

# 3. System Architecture Overview

The system consists of two major layers:

## 3.1 Backend (Integration Only)

- EV Firmware (Generates telemetry)
- Firmware REST API
- WebSocket Server

Backend acts only as a data provider.

---

## 3.2 Frontend Application (Core System)

The frontend is divided into multiple layers:

1. Data Access Layer
2. State Management Layer
3. Vehicle Detection Module
4. Configuration Layer
5. Dashboard Engine
6. Tile Registry
7. Presentation Layer

---

# 3.3 Architecture Diagrams Explanation

This project includes three major architecture diagrams:

### 1.Development Workflow Diagram

This diagram illustrates the complete phase-wise implementation process from project setup to optimization and testing. It highlights how the system is built incrementally using React, Zustand, Axios, and WebSocket.

### 2.Enterprise Frontend Architecture Diagram

This diagram shows the internal layered structure of the frontend engine. It clearly separates:

- Data Access Layer
- State Management Layer
- Configuration Layer
- Dashboard Engine
- Tile Registry
- Presentation Layer

It also emphasizes that the backend acts only as a data source.

### 3. Layered Start-to-End Flow Diagram

This diagram represents the complete runtime execution flow from EV firmware to final UI rendering, showing how telemetry data travels through each architectural layer.

These diagrams collectively provide a comprehensive visualization of system design, development workflow, and runtime execution.

# 4. Detailed Frontend Architecture

---

## 4.1 Data Access Layer

Responsibilities:

- Fetch telemetry via Axios

- Handle WebSocket connection
- Normalize incoming data
- Implement retry logic
- Fallback to polling

---

## 4.2 State Management Layer

Implemented using Zustand.

Stores:

- Current vehicle ID
- Telemetry data
- UI state
- Connection status

Implements:

- Observer Pattern
- Selective Re-render
- Memoization

---

## 4.3 Vehicle Detection Module

Determines active vehicle using:

- URL parameters

- Local storage
- API response
- Default fallback

Loads corresponding configuration.

---

## 4.4 Configuration Layer (Core of System)

Uses:

vehicleConfigs.json

Defines:

- Layout structure
- Theme
- Tile types
- Tile positions
- Data mapping (dataKey)

Key Principle:

Changing configuration automatically updates UI without modifying engine code.

---

## 4.5 Dashboard Engine

Single dynamic component:

- Reads selected vehicle configuration
- Loops through config.tiles

- Dynamically renders components
- No hardcoded dashboards

---

## 4.6 Tile Registry (Factory Pattern)

Maps:

tile.type → Component

Examples:

- GaugeTile
- MetricTile
- ChartTile
- MapTile
- StatusTile

Ensures:

- Reusability
- Scalability
- Clean separation

---

## 4.7 Presentation Layer

Handles:

- Responsive grid layout

- Theme application
- Mobile/tablet/desktop support
- Final UI rendering

---

# 5. Development Workflow (Build Phases)

---

## Phase 1: Project Setup

- Create React + Vite project
- Install dependencies
- Setup Tailwind CSS

---

## Phase 2: Static Dashboard UI

- Create basic layout
- Implement grid system
- Design basic tiles

---

## Phase 3: Configuration System

- Create vehicleConfigs.json
- Define layout & tile schema

- Connect UI to config

---

## Phase 4: Tile Component Library

- Create reusable tile components
- Ensure data-agnostic design

---

## Phase 5: Dashboard Engine Implementation

- Build dynamic rendering logic
- Implement Tile Registry

---

## Phase 6: State Management

- Setup Zustand store
- Connect tiles to store

---

## Phase 7: API Integration

- Connect REST API
- Update state with telemetry

---

## Phase 8: Real-Time Integration

- Implement WebSocket
- Update store on incoming data
- Enable selective re-render

---

## Phase 9: Optimization & Testing

- Improve performance
- Add error handling
- Responsive testing

---

# 6. Data Flow (Start to End)

EV Firmware
↓
Firmware REST API
↓
Frontend Data Access Layer
↓
Data Normalization
↓
Zustand Store
↓
Vehicle Detection
↓
Load Configuration
↓
Dashboard Engine
↓
Tile Registry
↓
Reusable Tiles

↓
Rendered UI

---

# 7. Real-Time Flow

WebSocket Server
↓
WebSocket Client
↓
Normalize Data
↓
Zustand Store Update
↓
Selective Re-render
↓
Live Dashboard Update

---

# 8.Non-Functional Requirements

## Performance

- Initial load < 3 seconds
- Frame render < 16ms
- Selective component re-render

## Scalability

- Supports 10+ vehicles
- Add new vehicle via config only
- Zero hardcoding

### Reliability

- Retry logic
- Polling fallback
- Error boundaries

### Maintainability

- Clean architecture
- Modular components
- Separation of concerns

---

## 9. Key Strengths of the System

- Configuration-driven UI
- No hardcoded dashboards
- Real-time update capability
- Reusable tile architecture
- Scalable multi-vehicle support
- Enterprise-level design principles

---

## 10. Conclusion

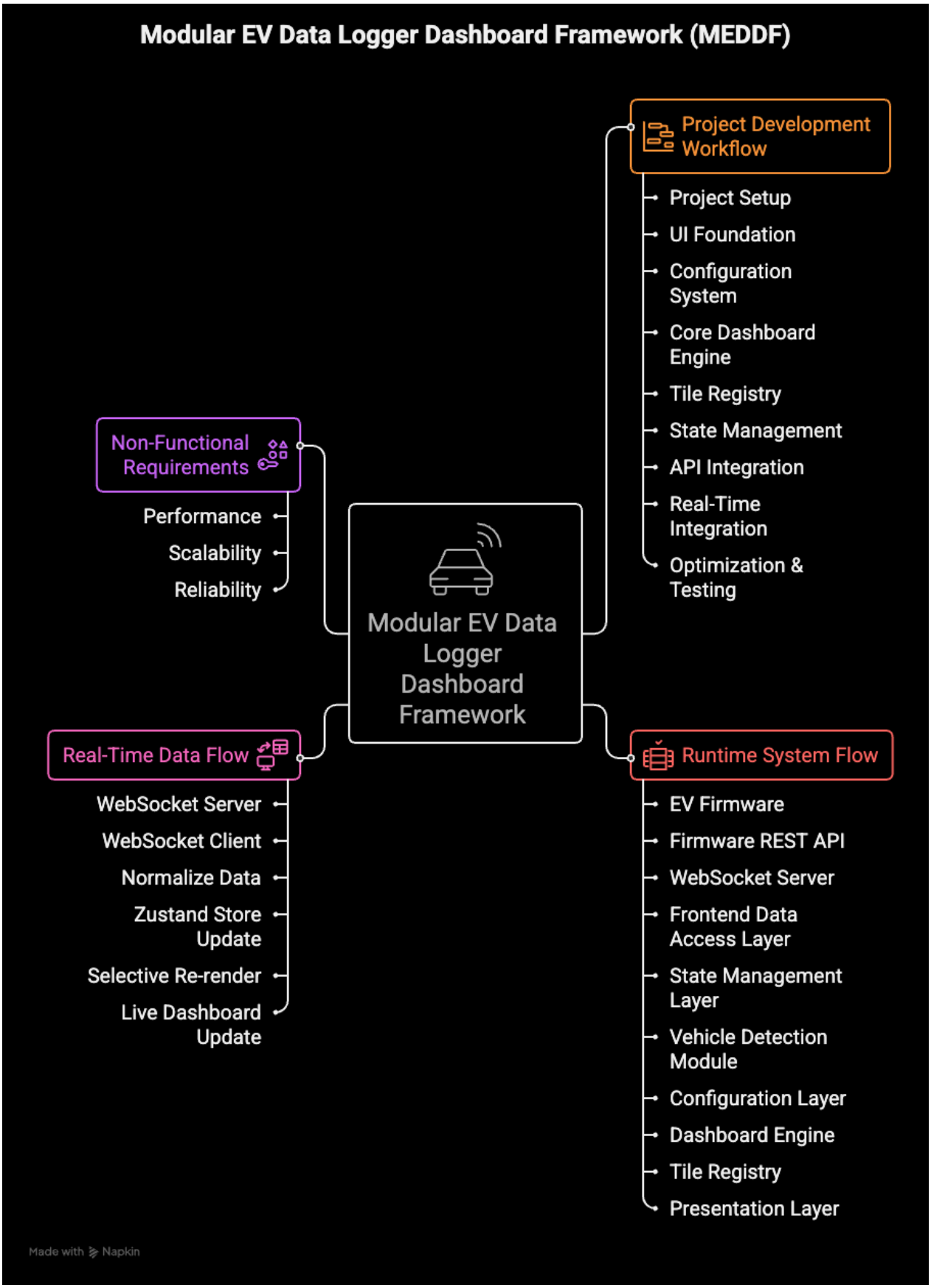
The MEDDF project is a scalable, configuration-driven frontend dashboard engine designed to dynamically render EV dashboards using modern frontend technologies.

It ensures:

- High performance
- Scalability
- Maintainability
- Clean architecture
- Real-time telemetry visualization

**Folder structure**

```
src/
├── api/              → Handles REST & WebSocket communication
│   ├── apiClient.js
│   ├── websocket.js
│
├── store/            → Zustand centralized state
│   ├── useVehicleStore.js
│
├── config/           → Configuration-driven structure
│   ├── vehicleConfigs.json
│
├── engine/           → Core dynamic rendering engine
│   ├── DashboardEngine.jsx
│   ├── TileRegistry.js
│
├── components/       → Reusable UI components
│   ├── tiles/
│       ├── GaugeTile.jsx
│       ├── MetricTile.jsx
│       ├── ChartTile.jsx
│
├── pages/            → Page-level components
│   ├── Dashboard.jsx
│
├── App.jsx           → Application entry
```


Modular EV Data Logger Dashboard Framework (MEDDF)
Modular EV Data Logger Dashboard Framework
Project Development Workflow
Project Setup
UI Foundation
Configuration System
Core Dashboard Engine
Tile Registry
State Management
API Integration
Real-Time Integration
Optimization & Testing
Non-Functional Requirements
Performance
Scalability
Reliability
Real-Time Data Flow
WebSocket Server
WebSocket Client
Normalize Data
Zustand Store Update
Selective Re-render
Live Dashboard Update
Runtime System Flow
EV Firmware
Firmware REST API
WebSocket Server
Frontend Data Access Layer
State Management Layer
Vehicle Detection Module
Configuration Layer
Dashboard Engine
Tile Registry
Presentation Layer
Made with Napkin

# Enterprise Configuration-Driven Frentend Dashboard Engine Architecture – MEDDF

## SECTION 1: EXTERNAL & BACKEND (INTEGRATION ONLY)

EV Firmware
- Telemery Data
- DTCS, Status

→

Firmware REST API
- GET /vehicle:jd
- GET /vehicle;id/realtime

→

Wosbsscot Server
- Real-time stream
- Falffbak to le/id/realtime

→

Wosbsspot Server
- Real-time stream
- Falfbask to Polling

Existing Backend – Data Source Only

## SECTION 2: FRONTEND APPLICATION (MAIN SYSTEM)

A. Data Access Layer
- Axios for REST
- Waseepot Client

→

B. State Management Layer
- Zustand Store
- Observer Pattern
- Selective Re-render
- Memoization

→

D. Configuration Layer
(CORE OF SYSTEM)
- layout
- tiles
- datatfoted dashping

Changing configuration automaicelly updates UI

B. State Management Layer
- Zustand Store
- Data Normiization
- Local Storage
- Retry Logic
- Polling Fallivak

→

C. State Detection Module
- layout
- theme
- Selective Re-render
- tiles
- dataKey mapping

→

E. Dashboard Engine
- Dynamic rending,jxs
- (SINGLE DYNAMIC
- No hardcted dashboards)

G. Presentation Layer
- GaugeTile
- MeticTile ChartI
- MapTile
- TsatusTile

→

F. Tile Registzry Fagine
- Dynamd Store Update
- No hardocted Deskotop

→

G. Presentation Layer
- Taiwlond CSS
- MetrtTile
- Theme Engine
- Mobile / Tablet / Desktop

Live Dashboard Update

## SECTION 3: REAL-TIME DATA FLOW (SIDE DIAGRAM)

Woshboat Client

↓

Normlize Data

↓

Zustanize Update

↓

Selective Re-render
(Observer Pattern)

↓

UI Rendering

## SECTION 4: NON-FUNCTIONAL REQUIREMENTS

| Initial Load | Frame Render | Vehicle Support | | |
|---|---|---|---|---|
| < 3 seccods | < 16ms | < 16ms | - Add via config only<br>- Zero hardcocing | - Zero hacnfig only<br>- Production-ready |

# MEDDF: From EV Data to Dynamic Dashboards

| | |
|---|---|
| EV Firmware Layer | Generates telemetry data |
| Existing Backend Layer | Provides data via REST API and WebSocket |
| Frontend Application Layer | Core of the system |
| Data Access Layer | Handles API calls and WebSocket communication |
| State Management Layer | Centralized store for application state |
| Vehicle Detection Module | Identifies the current vehicle |
| Configuration Layer | Manages vehicle configurations |
| Dashboard Engine Layer | Dynamically renders the dashboard |
| Tile Registry | Maps tile types to components |
| Presentation Layer | Styles and displays the dashboard |

Made with Napkin